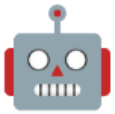


# ML Classic

{ وَاتَّقُوا اللَّهَ وَيُعَلِّمُكُمُ اللَّهُ وَاللَّهُ بِكُلِّ شَيْءٍ عَلِيمٌ }

👨‍💻 **github**: elma-dev

🔗 **linkden**: EL MAJJODI Abdeljalil

## ▼ How To Chose The Best Algorithm?

### 1- Analyse Your Data

Before you start thinking about the different algorithms, you need to familiarize yourself with your data. A simple way to do that is to **visualize the data** and try to find patterns within it, try to observe it's behavior, and, most importantly of all, its size.

### 2- Size Of Data

For example, for *small* training datasets, algorithms with ***high*** bais/ ***low*** variance classifiers will work better than ***low*** bias/ ***high*** variance classifiers. So, for small training data, ***Naïve Bayes*** will perform better than ***kNN***.

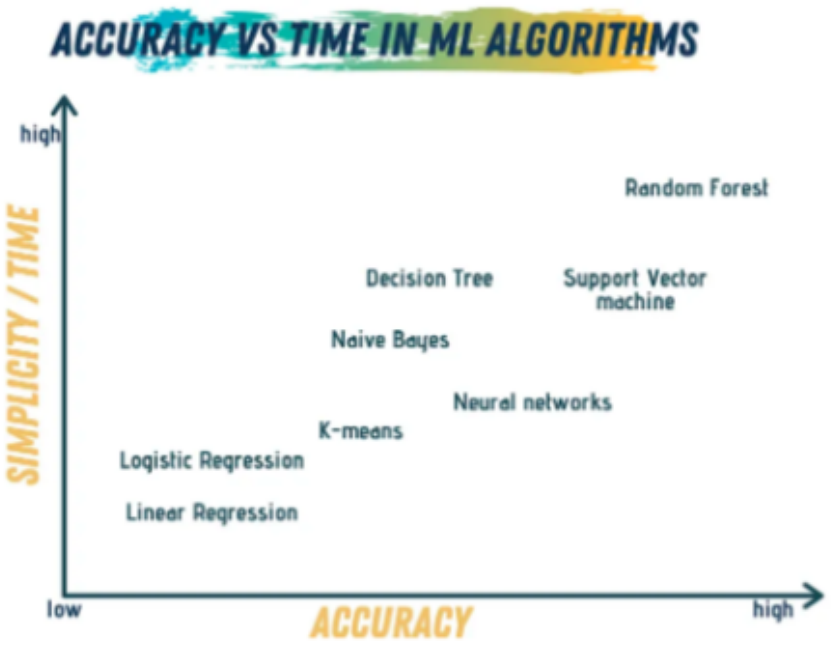
### 3- The Characteristics of Data

This means how your data is formed. Is your data linear? Then maybe a **linear** model will fit it best, such as ***regressions* — linear and logistic — or *SVM*** (support vector machine). However, if your data is **more complex**, then you need an algorithm like **R*andom forest***.

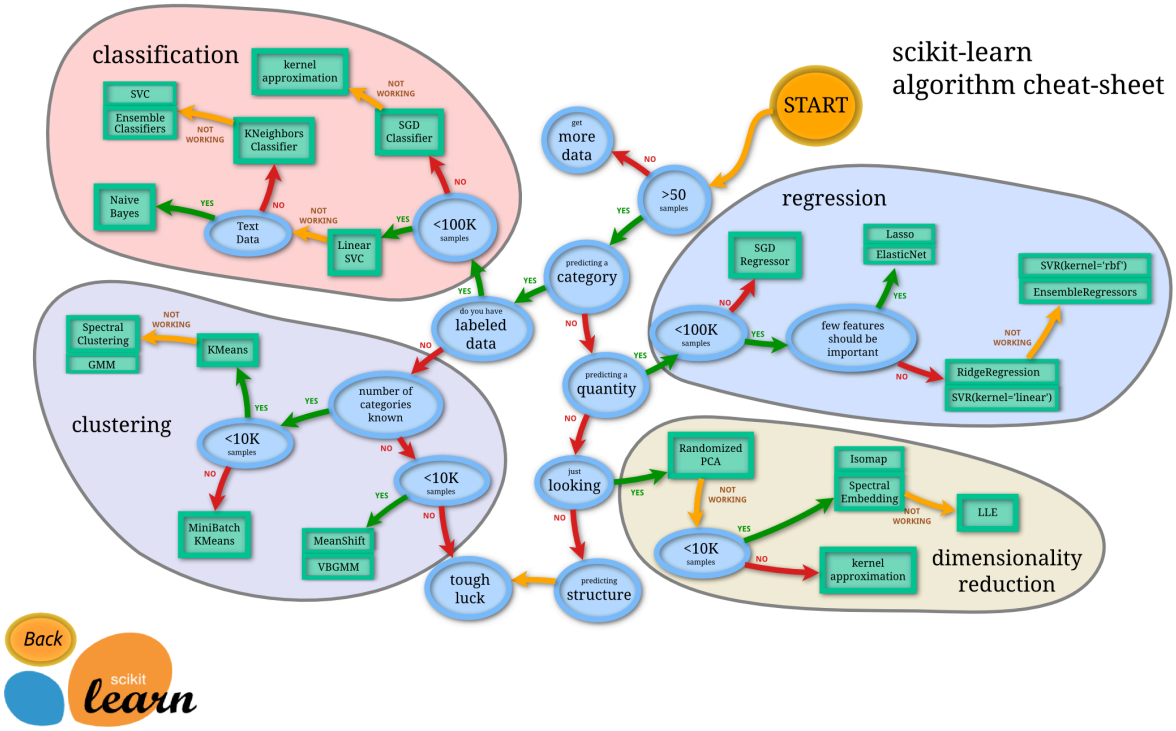
### 4- Accuracy

The accuracy of a model refers to its ability to predict an answer from a given observation set close to the correct response for that observation set.

## 5- Accuracy Vs Time




## 6- Some Examples/sklearnMap




## COMMON MACHINE LEARNING ALGORITHMS

| ALGORITHM | USAGE EXAMPLES |
|---|---|
| Linear Regression | Time between two locations. Predicting sales of a product. Improve yearly revenue projections. |
| Logistic Regression | Predicting the Customer rate of attrition. Fraud Detection. Measuring campaigns effectiveness. |
| Decision Tree | Investment decisions. Banks loan estimations. Lead qualifications |
| Naive Bayes | Sentiment analysis. Text classification. Recommendation systems. Face recognition |
| K-means | Grouping something or concentrating on particular groups, Contact tracing. |
| Support Vector machine | Detecting persons with common diseases such as diabetes. Recognize hand-written text. Product price prediction. |
| Random Forest | Predict high risks patients. Predict parts failures in manufacturing. |

# ▼ Effective Visualizations

### "Data is a story told in numbers, visualizing it is how you're telling the story."

## Tip №1: Simple is always better

- The goal of using visualization is to make information easier to read and understand by others. So, having **complex**, **crowded visualization** is something to be **avoided**.

## Tip №2: Choose the right chart type

1. If you have ***categorical data***, use a **bar chart** if you have more than 5 categories or a **pie chart** otherwise.
2. If you have ***nominal data***, use **bar charts** or **histograms** if your data is discrete, or **line/ area charts** if it is continuous.
3. If you want to show the ***relationship between values*** in your dataset, use a **scatter plot, bubble chart**, or ***line charts***.
4. If you want to ***compare values***, use a **pie chart** — for relative comparison — or **bar charts** — for precise comparison.

---

# ▼ How to Choose a Chart Type?

- **What is your data type?**

There are several types of data, describe, continuous, qualitative, or categorial. You can use the kind of data to eliminate some chart types. For example, if you have continuous data, a bar chart may not be the best choice; you may need to go with a line chart instead.

Similarly, if you have categorical data, then using a bar chart or a pie chart may be a good idea. You probably will not want to use a line chart with categorical data, because by definition, you can't have continuous categories. The has to be a discrete finite amount of categories.

- **The top 7 used chart types:**

1. **Pie Chart**

2. **Line Chart**
3. **Scatter Plot**
4. **Area Chart**
5. **Bar Chart**
6. **Bubble Chart**
7. **Combined Chart**

- **Chart selection tips:**

1. If you have categorical data, use a bar chart if you have more than 5 categories or a pie chart otherwise.
2. If you have nominal data, use bar charts or histograms if your data is discrete, or line/ area charts if it is continuous.
3. If you want to show the relationship between values in your dataset, use a scatter plot, bubble chart, or line charts.
4. If you want to compare values, use a pie chart — for relative comparison — or bar charts — for precise comparison.
5. If you want to compare volumes, use an area chart or a bubble chart.
6. If you want to show trends and patterns in your data, use a line chart, bar chart, or scatter plot.

---

# ▼ Feature Scaling

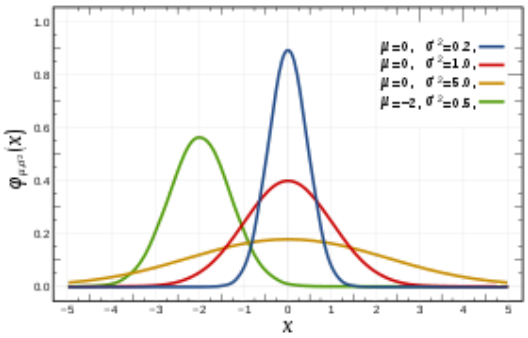
**Scale generally means to change the range of the values.**

- **Normalization :**

used when distribution of data does not follow Gaussian distribution like neural networks

- **Standardization**

is applied when gaussian distribution is followed e.g. Principal Component Analysis (PCA).




- **MinMaxScaler**

is used to have a light touch on features

- **RobustScaler**

is used to reduce the influence of outliers.

### 👀Before or after splitting?

**(Rep from** https://datascience.stackexchange.com/**)**

Normalization across instances should be done after splitting the data between training and test set, using only the data from the training set.

This is because the test set plays the role of fresh unseen data, so it's not supposed to be accessible at the training stage. Using any information coming from the test set before or during training is a potential bias in the evaluation of the performance.

When normalizing the test set, one should apply the normalization parameters previously obtained from the training set as-is. *Do not* recalculate them on the test set, because they would be inconsistent with the model and this would produce wrong predictions.

---

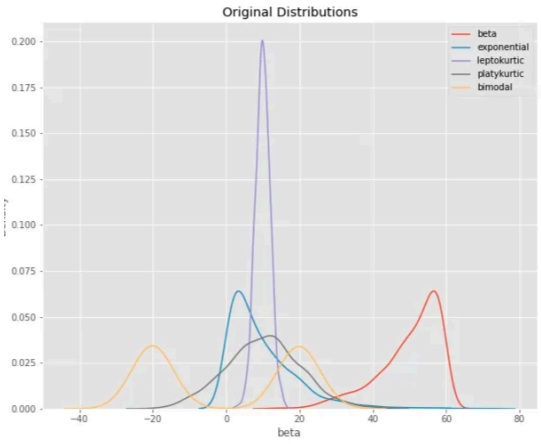
# ▼ DATA Preprocessing (Some Tec)

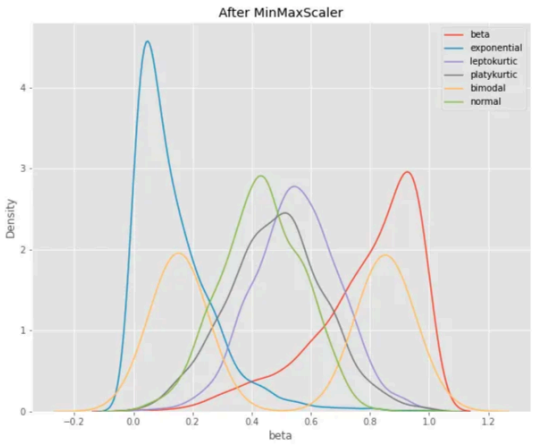
### When to use *MinMaxScaler*, *RobustScaler*, *StandardScaler*, and *Normalizer*

> **MinMaxScaler :**

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

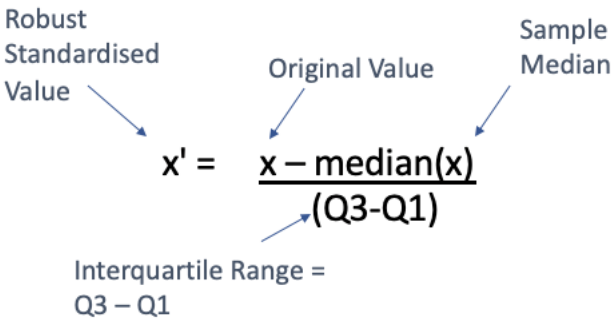
- **MinMaxScaler preserves the shape of the original distribution. It doesn't meaningfully change the information embedded in the original data.**
- **Note that MinMaxScaler doesn't reduce the importance of outliers**
- **The default range for the feature returned by MinMaxScaler is 0 to 1.**

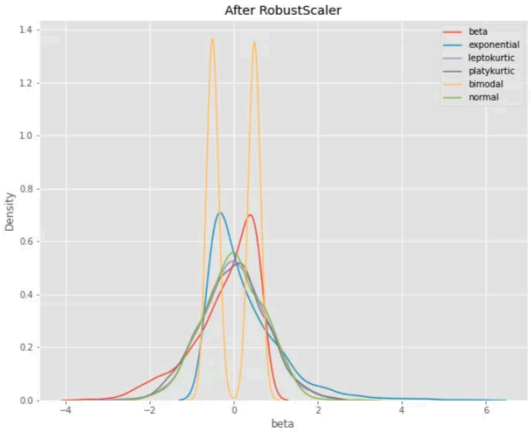



- **how the features are all on the same relative scale. The relative spaces between each feature's values have been maintained**
- **MinMaxScaler isn't a bad place to start, unless you know you want your feature to have a normal distribution or you have outliers and you want them to have reduced influence.**

### RobustScaler:



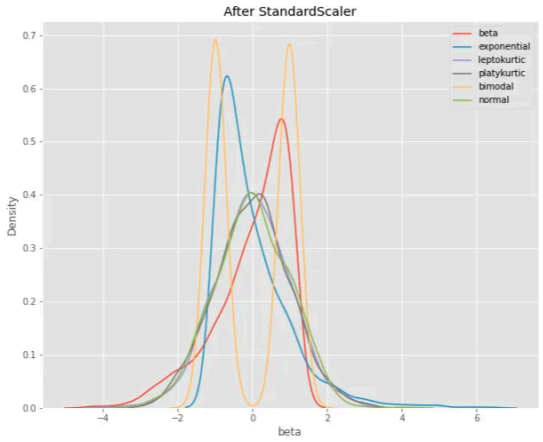




**Like MinMaxScaler, our feature with large values — *normal-big*— is now of similar scale to the other features. Note that RobustScaler does not scale the data into a predetermined interval like MinMaxScaler. It does not meet the strict definition of *scale* I introduced earlier.**

- **Use RobustScaler if you want to reduce the effects of outliers, relative to MinMaxScaler.**

## StandardScaler:

$$\text{Standardization:}$$
$$z = \frac{x - \mu}{\sigma}$$
$$\text{with mean:}$$
$$\mu = \frac{1}{N}\sum_{i=1}^{N}(x_i)$$
$$\text{and standard deviation:}$$
$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$



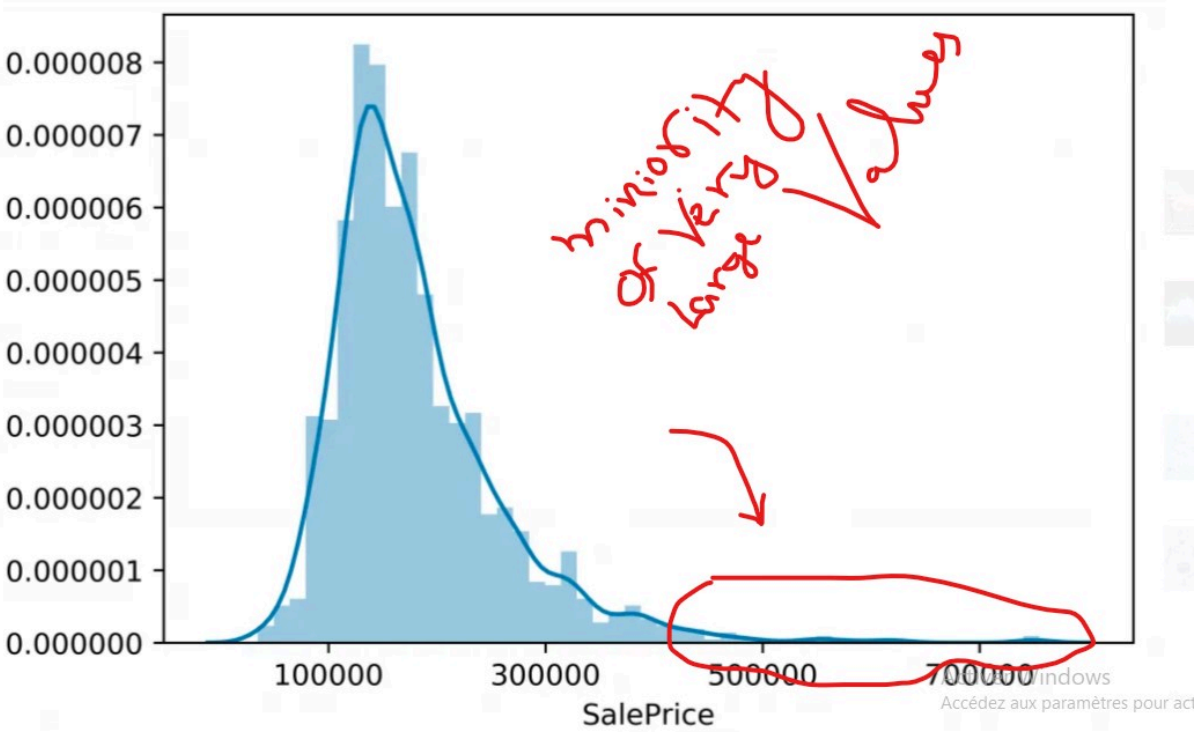

- **StandardScaler does not meet the strict definition of *scale* I introduced earlier.**
- **StandardScaler makes the mean of the distribution approximately 0.**
- **in the plot above, you can see that all four distributions have a mean close to zero and unit variance. The values are on a similar scale, but the range is larger than after MinMaxScaler.**

> **Conclusion**

| A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|
| **Preprocessing Type** | **Scikit-learn Function** | **Range** | **Mean** | **Distribution Characteristics** | **When Use** | **Definition** | **Notes** |
| Scale | MinMaxScaler | 0 to 1 default, can override | varies | Bounded | When want a light touch. | Add or substract a constant. Then multiply or divide by another constant. MinMaxScaler subtracts the mimimum value in the column and then divides by the difference between the original maximum and original minimum. | Preserves the shape of the original distribution. Doesn't reduce the importance of outliers. Least disruptive to the information in the original data. Default range for MinMaxScaler is 0 to 1. |
| Standardize | RobustScaler | varies | varies | Unbounded | Use if have outliers and don't want them to have much influence. | RobustScaler standardizes a feature by removing the median and dividing each feature by the interquartile range. | Outliers have less influence than with MinMaxScaler. Range is larger than MinMaxScaler or StandardScaler. |
| Standardize | StandardScaler | varies | 0 | Unbounded, Unit variance | When need to transform a feature has zero mean and unit standard deviation. It's my go-to. | StandardScaler standardizes a feature by removing the mean and dividing each value by the standard deviation. | Results in a distribution with a standard deviation equal to 1 (and variance equal to 1). If you have outliers in your feature (column), normalizing your data will scale most of the data to a small interval. |
| Normalize | Normalizer | varies | 0 | Unit norm | Rarely. | An observation (row) is normalized by applying l2 (Euclidian) normalization. If each element were squared and summed, the total would equal 1. Could also specify l1 (Manhatten) normalization. | Normalizes each sample observation (row), not the feature (column)! |

# ▼ Transforming Skewed Data

> ***skew is the degree of distortion from a normal distribution.***




▼ **Why do we care if the data is skewed?**

**the model will be trained on a much larger number of moderately "priced homes" (in this example), and will be less likely to successfully predict the price for the most expensive houses.**

▼ **determine if the variable is skewed**

**We can objectively determine if the variable is skewed using the *Shapiro-Wilks* test.**

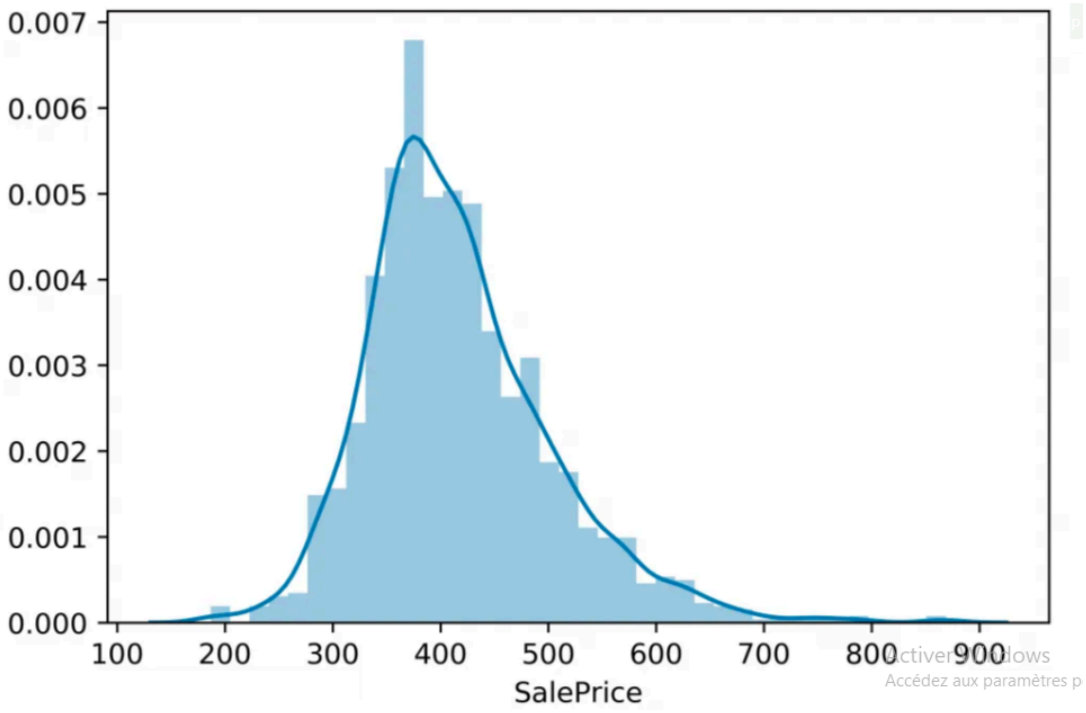
```
1  from scipy.stats import shapiro
2  shapiro(x)
```

- **The null hypothesis for this test is that the data is a sample from a normal distribution**
- **p-value less than 0.05 indicates significant skewness.**

**More convenient way of evaluating skewness is with pandas' *".skew"* method.**

## ▼ Transformation

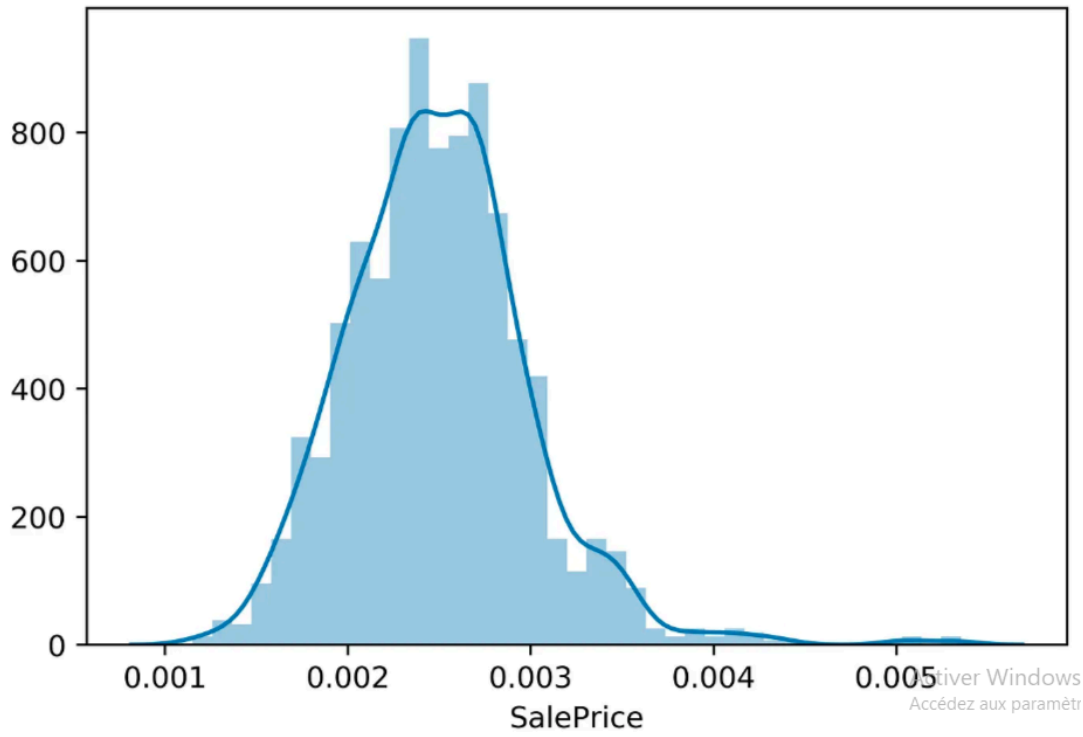
### ▼ Square Root Transformation



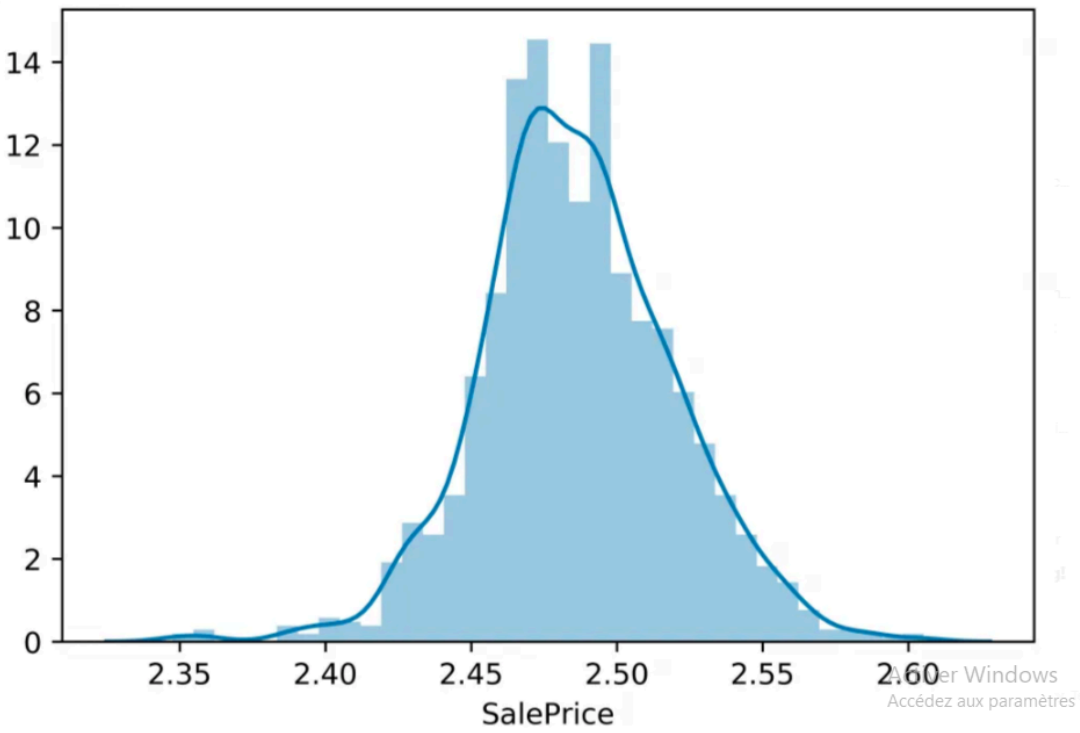

**After transforming, the data is definitely less skewed, but there is still a long right tail.**

**Still not great, the above distribution is not quite symmetrical.**

### ▼ Reciprocal Transformation (1/x)



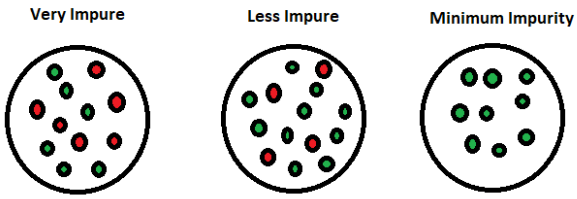

▼ Log Transformation




**The log transformation seems to be the best, as the distribution of transformed sale prices is the most symmetrical.**

# ▼ Entropy

***Entropy*** **is an information theory metric that measures the impurity or uncertainty in a group of observations.**




- **Using the Entropy we can define the root of the decision tree.**

### ▼ How calculate Entropy?

**Consider a dataset with N classes. The entropy may be calculated using the formula below:**

$$\mathrm{H}(X) = -\sum_{i=1}^{n} \mathrm{P}(x_i) \log \mathrm{P}(x_i)$$

**Example:**

**Let's try to calculate the E in this dataset:**

| | outlook | temp | humidity | windy | play |
|---|---|---|---|---|---|
| 0 | sunny | hot | high | 0 | 0 |
| 1 | sunny | hot | high | 1 | 0 |
| 2 | overcast | hot | high | 0 | 1 |
| 3 | rainy | mild | high | 0 | 1 |
| 4 | rainy | cool | normal | 0 | 1 |
| 5 | rainy | cool | normal | 1 | 0 |
| 6 | overcast | cool | normal | 1 | 1 |
| 7 | sunny | mild | high | 0 | 0 |
| 8 | sunny | cool | normal | 0 | 1 |
| 9 | rainy | mild | normal | 0 | 1 |
| 10 | sunny | mild | normal | 1 | 1 |
| 11 | overcast | mild | high | 1 | 1 |
| 12 | overcast | hot | normal | 0 | 1 |
| 13 | rainy | mild | high | 1 | 0 |

- **This dataset contains 2 categories (Yes/No) and N=14 .**

$$P(Yes = 1) = len(Yes)/N = 9/14$$

$$P(No = 0) = len(No)/N = 5/14$$

$$E(Data) = -P(Yes)log(P(Yes)) - P(No)log(No)$$

$$E(Data) = -9/14 * log(9/14) - 5/14 * log(5/14) = 0.94$$

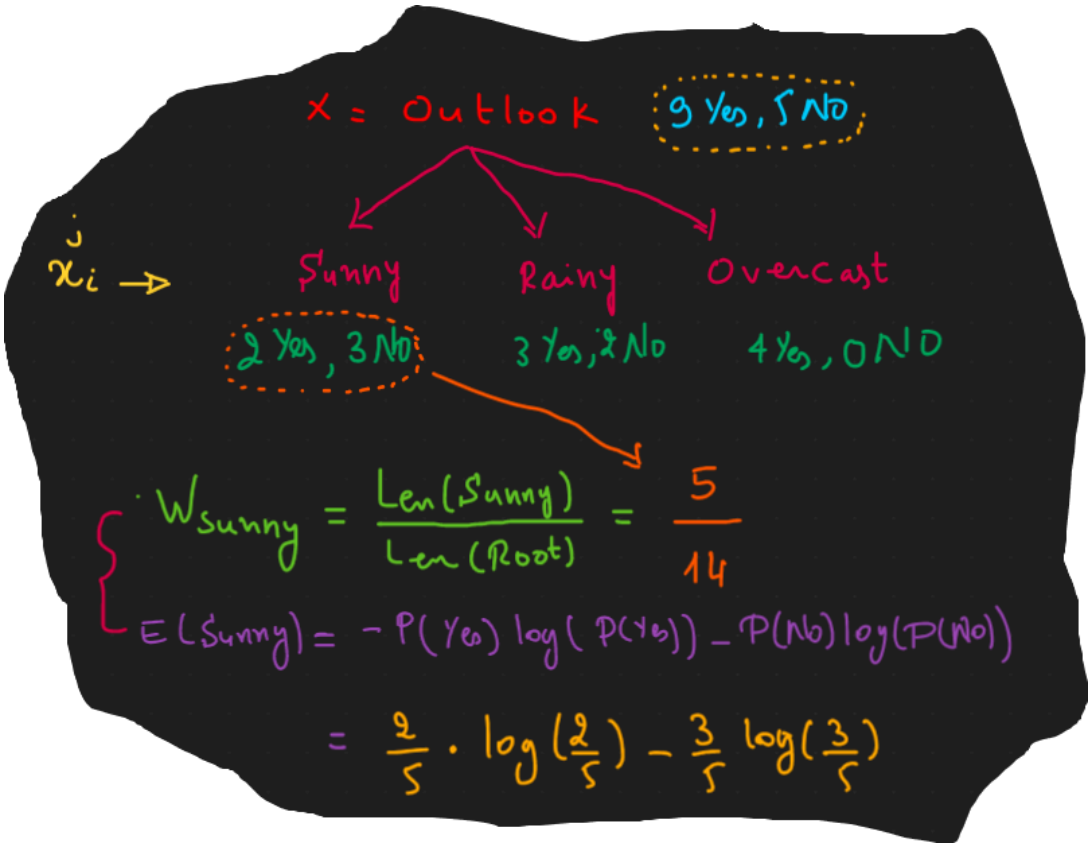
▼ **Gain**

✔️ **The feature wight the biggest gain will be the root of the decision tree.**

➡️ **after we calculate the Entropy of the dataset now we calculate the *Gain information* with this operation:**

$$G(X = feature) = E(data) - (\sum_{i=1}^{n} W_{x_i^j.right} E(x_i^j))$$

- **for example l'ets calculate the Gain of X=outlook :**




*→ in this image i calculate juste W(sunny) and E(sunny), after you follow the same to calculate the others then you can get the value of Gain.*

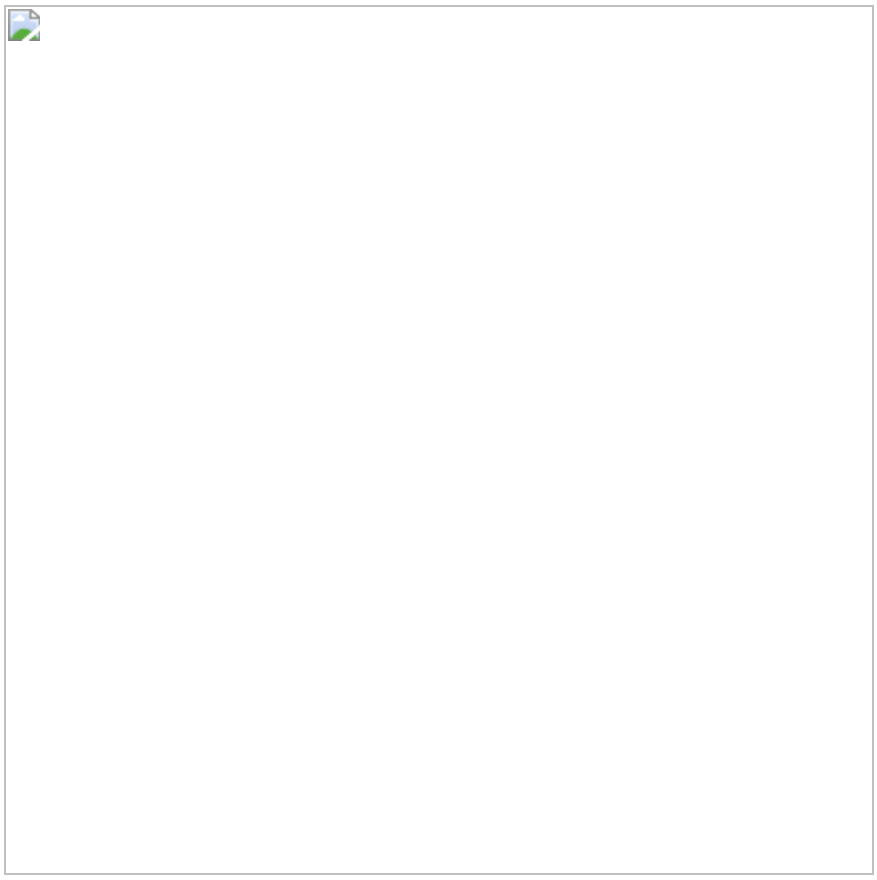
- **Entropy(outlook = sunny) = -2/5 log2(2/5)-3/5log2(3/5) = 0.971**
- **Entropy(outlook = overcast) = -4/4 log2(4/4) = 0**
- **Entropy(outlook = rainy) = -3/5 log2(3/5)-2/5log2(2/5) = 0.971**

**and the gain is:**

- **Gain(outlook) = 0.94-[5/14 * 0.971+ 4/14 * 0 + 5/14 * 0.971] = 0.247**

*→ you should to calculate the gain of each features in your dataset following the same process then the feature with the highest gain is the root.*

*→ after get the root you try to extract the sub tree with the other feature like in the next example:*



## ▼ Split Data &  Why do we need to add cross-validation operation :

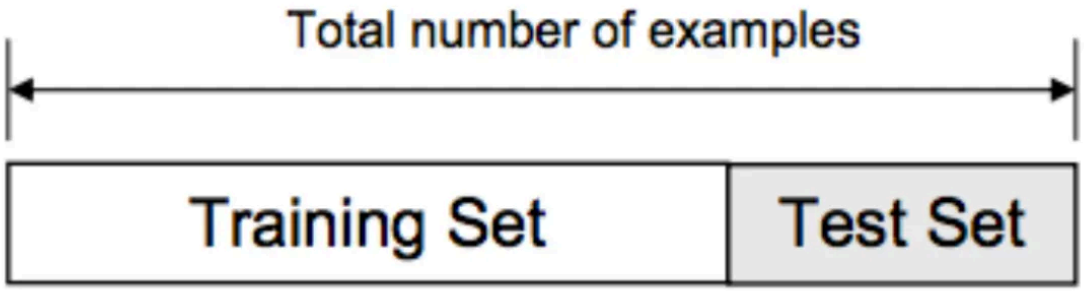
```
Main Question "Problem":
But train/test split does have its dangers – what if the s
plit we make isn't random?

    /**
Cross Validation Allow Us To compare different machine lea
rning methods and get a sens
of how well they will work in practice
**/
```

- **It is the process by which the machine learning models are evaluated on a separate set known as validation set or hold-out set with which *the best hyper-parameters are found*, so that we get the optimal model, that can be used on future data and which is capable of yielding the best possible predictions**

→ One way of doing this is to split our dataset into 3 parts: **Training Set**, **Validation** or **Hold-Out set** and the **Test Set**.

## ▼ Training , Validation and Testing Split / Cross Validation




**Training Set:** The part of the Dataset on which the model is trained

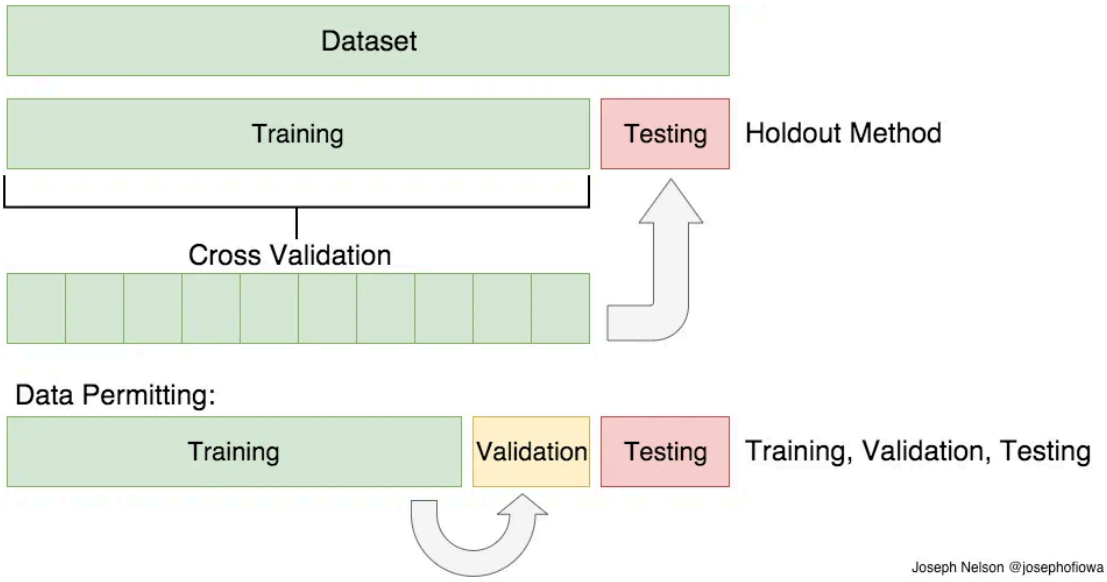
**Validation Set:** The trained model is then used on this set to predict the targets and the loss is noted. The result is compared to the training set results to check for overfitting or underfitting and this is done repeatedly until certain optimal result is produced.

**basically, we are training the model on the training set. but, the hyper-parameters are continuously updated and trained again until the model performs best on the cross-validation set. *(By performing best, we are trying to minimize the validation set while also preventing overfitting)***

**Test Set:** The fully trained model, *after being evaluated by the validation or hold-out set is used on the test* set to get the true test error and this error can be treated as a very good estimate of how the performance of the model will be on any new data

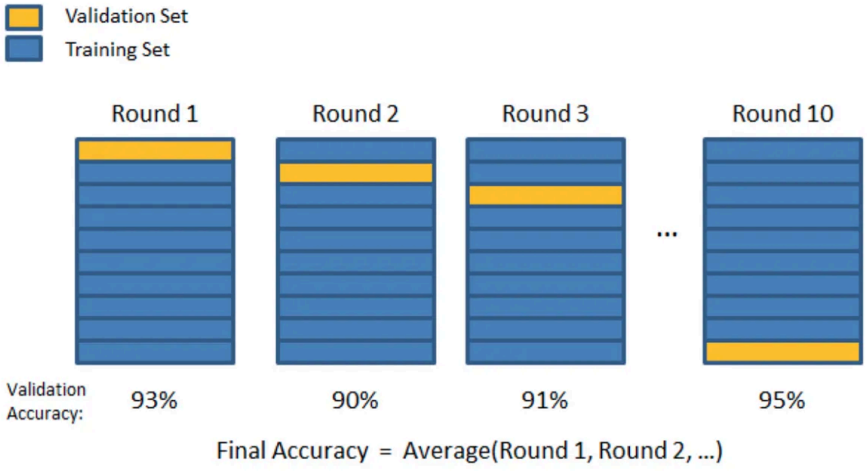
### ▼ How it's work ?

*It's very similar to train/test split, but it's applied to more subsets. Meaning, we split our data into k subsets, and train on k-1 one of those subset. What we do is to hold the last subset for test. We're able to do it for each of the subsets.*




# ▼ Cross Validation Methods:

## ▼ K-Folds Cross Validation



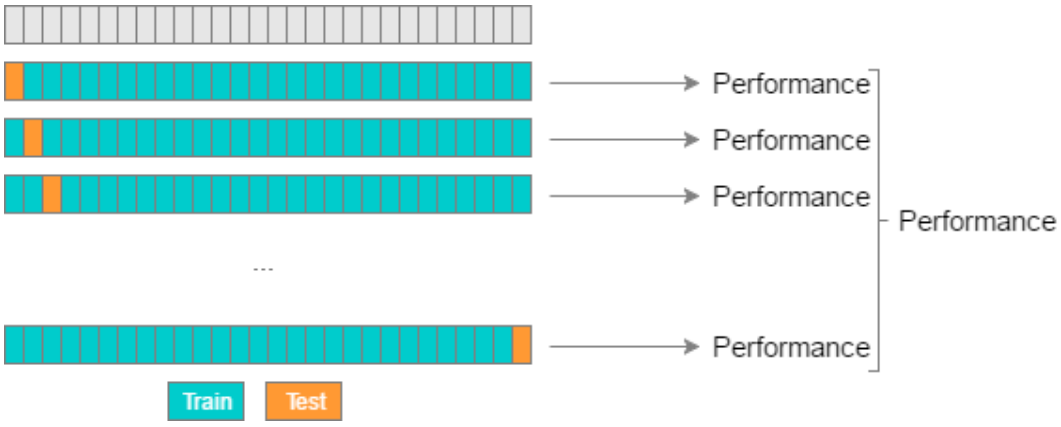

**In K-Folds Cross Validation we split our data into k different subsets (or folds). We use k-1 subsets to train our data and leave the last subset (or the last fold) as test data. We then average the model**

**against each of the folds and then finalize our model. After that we test it against the test set.**

```
from sklearn.model_selection import KFold # import
KFold
X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]]) # cr
eate an array
y = np.array([1, 2, 3, 4]) # Create another array
kf = KFold(n_splits=2) # Define the split - into 2
folds
kf.get_n_splits(X) # returns the number of splittin
g iterations in the cross-validator
print(kf)
KFold(n_splits=2, random_state=None, shuffle=False)
```

```
#Get Indexes
for train_index, test_index in kf.split(X):
 print("TRAIN:", train_index, "TEST:", test_index)
 X_train, X_test = X[train_index], X[test_index]
 y_train, y_test = y[train_index], y[test_index]
# OUTPUT
('TRAIN:', array([2, 3]), 'TEST:', array([0, 1]))
('TRAIN:', array([0, 1]), 'TEST:', array([2, 3]))
```

## ▼ Leave One Out Cross Validation



**In this type of cross validation, the number of folds (subsets) equals to the number of observations we have in the dataset. We then average ALL of these folds and build our model with the average. We then test the model against the last fold. Because we would get a big number of training sets (equals to the number of samples), this method is very computationally expensive and should be used on small datasets. If the dataset is big, it would most likely be better to use a different method, like kfold.**

## ▼ LveOneOut Sklearn

```
#Use LaveOneOut in sklearn
from sklearn.model_selection import LeaveOneOut
X = np.array([[1, 2], [3, 4]])
y = np.array([1, 2])
loo = LeaveOneOut()
loo.get_n_splits(X)
```

```
#Show The results
for train_index, test_index in loo.split(X):
   print("TRAIN:", train_index, "TEST:", test_inde
x)
   X_train, X_test = X[train_index], X[test_index]
   y_train, y_test = y[train_index], y[test_index]
   print(X_train, X_test, y_train, y_test)
```

```
#The results
('TRAIN:', array([1]), 'TEST:', array([0]))
(array([[3, 4]]), array([[1, 2]]), array([2]), arra
y([1]))
('TRAIN:', array([0]), 'TEST:', array([1]))
(array([[1, 2]]), array([[3, 4]]), array([1]), arra
y([2]))
```

### ▼ *cross_val_predict & cross_val_score methods in sklearn*

- ***cross_val_predict*** function to return the predicted values for each data point when it's in the testing slice.
- ***cross_val_score*** *return the score in each k-train*

```
# Necessary imports:
from sklearn.model_selection import cross_val_scor
e, cross_val_predict
```

```
scores = cross_val_score(model, df, y, cv=6) #cv=6
-> 6-folds df : dataframe model:estimator
```

```
# Make cross validated predictions
predictions = cross_val_predict(model, df, y, cv=6)
```

---

# ▼ Other

## Most Used Libs In ML

- [ ] Anaconda IDL
- [ ] Numpy
- [ ] Matplotlib
- [ ] Pandas
- [ ] Seaborn
- [ ] Plotly.express
- [ ] Yellowbrick

## Seaborn Libs

.pairplot() get all plot/correlation between features

## Data Analyse Trics

- [ ] Name Features: we need to **remove space**, **remove special characters**, and **set it uppercase**.
- [ ] To do this thing we can use many libs like **Pyjanitor (data.clean_names)**
- [ ] with categorical data with **describe()** we can get: **count, mod, freq**